Լաբորատոր աշխատանք 11
6 Խնայողության նպատակային հաշիվ

Աշխատանքի նպատակը

Ուսումնասիրել C++ լեզվում դասերի, օբյեկտների և արտաքին ֆունկցիաների կիրառումը: Ստեղծել `SavingsPlan` դաս, որը կպահպանի ամսական ներդրումների զանգվածը և նպատակային գումարը: Իրականացնել տվյալների վերլուծություն, պատճենի փոփոխություն և ներդրումների հերթականության հակադարձում:

Խնդրի պահանջները

1. Ստեղծել `SavingsPlan` դաս, որը կպարունակի ամսական ներդրումների տվյալները և նպատակային գումարը:
2. Տվյալները պահել `private` հասանելիության ներքո՝ օգտագործելով `std::vector<double>`:
3. Իրականացնել պարամետրերով կոնստրուկտոր, տվյալների ընթերցման մեթոդներ, `set(index, value)` և `print()` մեթոդները:
4. Իրականացնել երեք արտաքին ֆունկցիաները՝ `analyze()`, `changeCopy()` և `reversed()`՝ առանց `friend` կիրառելու:
5. Ստուգել տվյալների թույլատրելիությունը և ապահովել, որ սխալ տվյալների դեպքում օբյեկտի նախկին արժեքները պահպանվեն:

Առաջադրանք

Ստեղծել `SavingsPlan` դաս՝ պահպանելով ամսական ներդրումների գումարները և նպատակային գումարը՝ `p`:

Ծրագիրը պետք է՝

- Հաշվի բոլոր ներդրումների ընդհանուր գումարը:
- Գտնի առաջին ամսվա ինդեքսը, երբ կուտակային գումարը հասնում է նպատակային գումարին: Եթե չի հասնում, վերադարձնի `-1`:
- Հաշվի միջին ներդրումից փոքր ներդրումների քանակը:
- Փոփոխի ներդրման արժեքը միայն օբյեկտի պատճենում՝ առանց սկզբնական օբյեկտը փոխելու:
- Ստեղծի նոր օբյեկտ՝ ներդրումների հակառակ հերթականությամբ և նույն նպատակային գումարով:

```
#include <iostream>
#include <vector>
using namespace std;
class SavingsPlan {
private:
    vector<double> payments;
    double p;
```

```
public:
    SavingsPlan(const vector<double>& values, double target) {
        payments = values;
        p = target;
    }
    const vector<double>& getPayments() const {
        return payments;
    }
    double getP() const {
        return p;
    }
    bool set(int index, double value) {
        if (index < 0 || index >= payments.size() || value < 0)
            return false;
        payments[index] = value;
        return true;
    }
    void print() const {
        cout << "Payments: ";
        for (int i = 0; i < payments.size(); i++) {
            cout << payments[i] << " ";
        }
        cout << "\nTarget amount: " << p << endl;
    }
};
void analyze(const SavingsPlan& object) {
    const vector<double>& v = object.getPayments();
    double sum = 0;
    double average;
    double current = 0;
    int firstMonth = -1;
    int count = 0;
    for (int i = 0; i < v.size(); i++) {
        sum += v[i];
        current += v[i];

        if (firstMonth == -1 && current >= object.getP()) {
            firstMonth = i;
        }
    }
    average = sum / v.size();
    for (int i = 0; i < v.size(); i++) {
        if (v[i] < average) {
            count++;
        }
    }
    cout << "\nTotal savings: " << sum << endl;
    cout << "First month index: " << firstMonth << endl;
```

```
    cout << "Number of payments below average: " << count << endl;
}
void changeCopy(SavingsPlan object, int index, double value) {
    if (object.set(index, value)) {
        cout << "\nModified copy:\n";
        object.print();
    }
    else {
        cout << "Invalid index or value!" << endl;
    }
}
SavingsPlan reversed(const SavingsPlan& object) {
    vector<double> v;
    for (int i = object.getPayments().size() - 1; i >= 0; i--) {
        v.push_back(object.getPayments()[i]);
    }
    return SavingsPlan(v, object.getP());
}
int main() {
    int n;
    cout << "Enter number of payments (3-30): ";
    cin >> n;
    if (n < 3 || n > 30) {
        cout << "Invalid number!" << endl;
        return 0;
    }
    vector<double> payments(n);
    cout << "Enter payment amounts:\n";
    for (int i = 0; i < n; i++) {
        cin >> payments[i];
        if (payments[i] < 0) {
            cout << "Invalid value!" << endl;
            return 0;
        }
    }
    double p;
    cout << "Enter target amount: ";
    cin >> p;
    if (p <= 0) {
        cout << "Invalid target amount!" << endl;
        return 0;
    }
    SavingsPlan plan(payments, p);
    cout << "\nOriginal data:\n";
    plan.print();
    analyze(plan);
    changeCopy(plan, 1, 12000);
    cout << "\nOriginal object after modification:\n";
```

```
    plan.print();
    SavingsPlan result = reversed(plan);
    cout << "\nReversed payments:\n";
    result.print();
    return 0;
}
```